

逻辑回归 (Logistic Regression) 原理与实践详解

你的名字

2025 年 9 月 1 日

目录

1	引言：名为“回归”的分类器	2
2	核心思想：从线性预测到概率转换	2
3	数学原理解析	2
3.1	Sigmoid 函数	2
3.2	假设函数 (Hypothesis)	3
3.3	代价函数 (Cost Function)	3
3.4	优化算法：梯度下降	3
4	Python 应用实例	4
5	结论	5

1 引言：名为“回归”的分类器

逻辑回归是机器学习领域中最基础且最重要的算法之一。尽管其名称中带有“回归”，但它却是一个强大而经典的分类算法，主要用于处理二分类问题，并可以轻松扩展到多分类任务。

逻辑回归通过一个巧妙的数学变换，将线性回归的连续输出值映射到了一个表示概率的 $(0, 1)$ 区间内，从而实现了离散类别的预测。因其模型简单、计算速度快、可解释性强，至今仍在工业界和学术界被广泛应用，并且是深度学习中神经网络单元的基础。

2 核心思想：从线性预测到概率转换

场景比喻：预测学生考试能否通过

假设我们想根据学生的“学习时长”来预测他“能否通过考试”（通过 $=1$ ，不通过 $=0$ ）。

如果我们直接使用线性回归，可能会拟合出一条直线。但这条直线会产生问题：当学习时长很长时，预测值可能会超过 1；时长很短时，预测值可能为负数。这显然不符合“概率”的定义（概率必须在 0 到 1 之间）。

“预测一个学生有 120% 的概率通过考试，或者 -10% 的概率通过，这毫无意义。”

逻辑回归的核心思想就是解决这个问题。它分为两步：

1. **线性叠加**：首先，像线性回归一样，计算出一个线性的预测值 $z = \theta^T \mathbf{x}$ 。这个值可以是任意实数。
2. **概率转换**：然后，将这个值 z “喂”给一个特殊的函数——**Sigmoid 函数**。这个函数就像一个“概率转换器”，无论输入什么值，它都能将其“压缩”到 $(0, 1)$ 区间内，从而得到一个漂亮的概率值。

这个从线性预测到概率输出的转换，是理解逻辑回归的关键。

3 数学原理解析

3.1 Sigmoid 函数

Sigmoid 函数，也称为逻辑函数（Logistic Function），是逻辑回归的核心。其数学表达式为：

$$g(z) = \frac{1}{1 + e^{-z}}$$

该函数具有优美的 S 形曲线，能够将任意实数 $z \in (-\infty, +\infty)$ 映射到 $(0, 1)$ 区间。

3.2 假设函数 (Hypothesis)

逻辑回归的假设函数 $h_{\theta}(\mathbf{x})$ 将线性组合 $\theta^T \mathbf{x}$ 通过 Sigmoid 函数进行转换，其输出被解释为给定输入 $\mathbf{x}$ 时，样本类别为 1 的概率。

$$h_{\theta}(\mathbf{x}) = g(\theta^T \mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

这里的 $h_{\theta}(\mathbf{x})$ 代表了 $P(y = 1 | \mathbf{x}; \theta)$ 。同理， $P(y = 0 | \mathbf{x}; \theta) = 1 - h_{\theta}(\mathbf{x})$ 。

3.3 代价函数 (Cost Function)

为了训练模型，我们需要一个代价函数来衡量预测的准确性。对于逻辑回归，如果使用均方误差 (MSE) 作为代价函数，会导致函数变为非凸函数，不利于梯度下降法寻找全局最优解。因此，我们采用基于最大似然估计推导出的 ** 对数损失 (Log Loss) **，也称为 ** 交叉熵损失 (Cross-Entropy Loss) **。

对于单个样本，其代价定义为：

$$\text{Cost}(h_{\theta}(\mathbf{x}), y) = \begin{cases} -\log(h_{\theta}(\mathbf{x})) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

通俗解读：一个“智能”的惩罚机制

这个代价函数非常聪明：

- **当真实答案是 1 时：**如果你预测的概率 $h_{\theta}(\mathbf{x})$ 接近 1 (比如 0.99)， $-\log(0.99)$ 的值接近 0，惩罚极小。但如果你预测的概率接近 0 (比如 0.01)， $-\log(0.01)$ 的值会变得巨大，施以重罚！
- **当真实答案是 0 时：**情况正好相反。如果你预测的概率 $h_{\theta}(\mathbf{x})$ 接近 0，惩罚就小；如果接近 1，惩罚就大。

它不仅惩罚错误，还惩罚“不自信的正确”和“过度自信的错误”，引导模型做出既准确又自信的预测。

为了方便计算，上述分段函数可以写成一个紧凑的形式：

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(\mathbf{x}^{(i)}))]$$

其中 m 是样本总数。我们的目标就是找到一组参数 θ ，使得这个总代价函数 $J(\theta)$ 最小。

3.4 优化算法：梯度下降

我们使用梯度下降法 (Gradient Descent) 来最小化代价函数 $J(\theta)$ 。该算法通过迭代更新参数 θ 来实现：

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

其中 α 是学习率。经过数学推导，代价函数对参数 θ_j 的偏导数为：

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

这个形式与线性回归的梯度下降更新规则惊人地相似，使得算法实现非常高效。

4 Python 应用实例

以下代码使用 ‘scikit-learn’ 库实现逻辑回归，并对一个合成的二分类数据集进行分类和可视化。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_classification
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.metrics import classification_report
7
8 # 1. 生成合成数据集
9 # n_features=2 方便可视化
10 X, y = make_classification(n_samples=200, n_features=2, n_redundant=0,
11                             n_informative=2, random_state=1,
12                             n_clusters_per_class=1)
13
14 # 2. 划分训练集和测试集
15 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
16                                                         random_state=42)
17
18 # 3. 创建并训练逻辑回归模型
19 model = LogisticRegression()
20 model.fit(X_train, y_train)
21
22 # 4. 评估模型
23 y_pred = model.predict(X_test)
24 print("分类报告:")
25 print(classification_report(y_test, y_pred))
26
27 # 5. 可视化决策边界
28 plt.figure(figsize=(10, 6))
29
30 # 绘制数据点
31 plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap=plt.cm.RdYlBu,
32             edgecolor='k', label='测试数据')
```

```

31 # 创建网格来绘制决策边界
32 ax = plt.gca()
33 xlim = ax.get_xlim()
34 ylim = ax.get_ylim()
35 xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], 100),
36                       np.linspace(ylim[0], ylim[1], 100))
37 Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
38 Z = Z.reshape(xx.shape)
39
40 # 绘制决策边界和概率等高线
41 ax.contourf(xx, yy, Z, cmap=plt.cm.RdYlBu, alpha=0.3)
42 ax.set_title("逻辑回归决策边界可视化")
43 ax.set_xlabel("特征 1")
44 ax.set_ylabel("特征 2")
45 plt.show()

```

Listing 1: 使用逻辑回归进行二分类并可视化决策边界

5 结论

逻辑回归是连接线性模型与广义线性模型的桥梁。它通过 Sigmoid 函数引入非线性变换，将线性回归的预测能力从数值预测扩展到了概率预测和类别决策，同时保持了模型的简洁性与可解释性。作为许多更复杂算法（如神经网络）的基石，深刻理解逻辑回归的原理，对于每一位数据科学从业者都至关重要。